DATA STRUCTURE — TRIE

Tries.

From autocomplete to XOR tricks — one tree to rule strings & bits.

LECTURER
Ahmed Tawfik

TOPIC
Tries · Binary Tries · XOR

FORMAT
Interactive · 60–90 min

What we'll cover today.

1

Motivation

Three real problems no flat list solves cheaply: *autocomplete*, *wildcard search*, *spell-check*.

2

The Trie

Anatomy, two implementations (array vs map), insert & search step-by-step, complexity.

3

Applications

Prefix counting, wildcard match, longest common prefix, distinct substrings.

4

Binary Trie

When the alphabet is just `{0, 1}`: the secret weapon for XOR problems.

5

XOR Problems

Max XOR pair · Min XOR · Max XOR subarray · Count pairs < K · Count pairs = K.

SECTION ONE

Motivation.

Three problems where naïve scanning is too slow and a hash map gives the wrong answer.

Autocomplete.

User types `"alg"` — find all words starting with that prefix, instantly.

TRY IT — TYPE A PREFIX

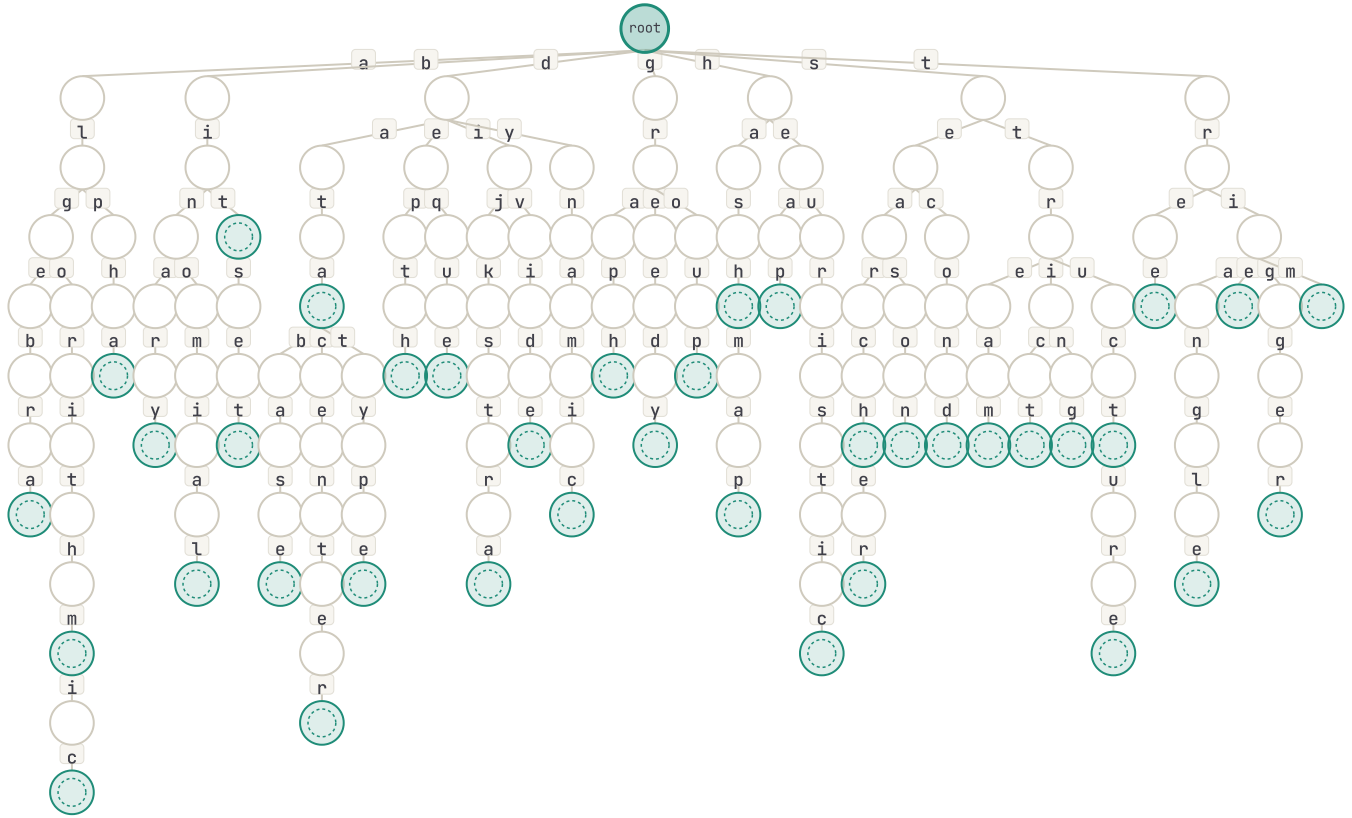
e.g. alg, str, bin...

38 words in dictionary

SUGGESTIONS

start typing...

TRIE VIEW — MATCH PATH HIGHLIGHTED



Walking down the trie from the root, one character per edge, lands you at the subtree of *all* completions in `O(|prefix|)`.

Wildcard match.

Match a pattern with ? (any one) and * (any sequence).

PATTERN

c?r*

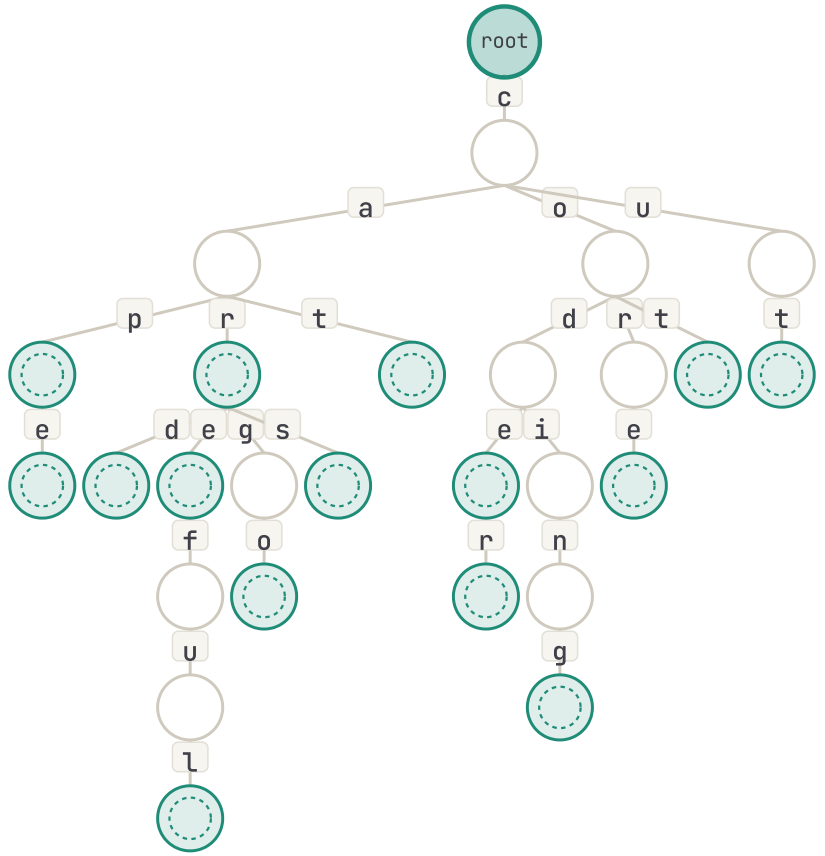
? any single char * any sequence

MATCHES (7)

car card care careful cargo cars core

On ? we recurse into *every* child; on * we recurse into every subtree at every depth. The trie prunes whole branches that can't match — much faster than scanning all words.

DICTIONARY TRIE



Spell-check.

Word not in dictionary — find words within k edits.

MISSPELL SOMETHING

algoritm

max edit distance: 1 2 3

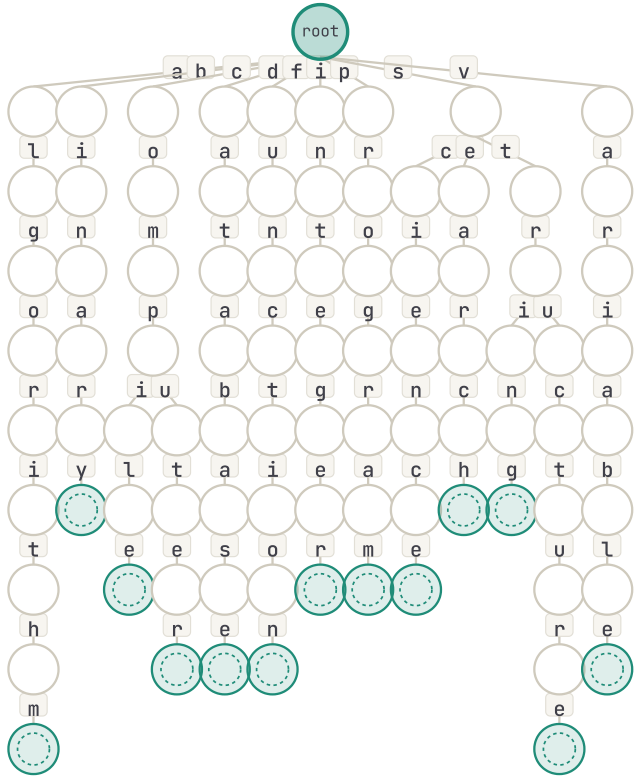
STATUS

✗ "algoritm" not found — did you mean…

SUGGESTIONS

algorithm edit dist 1

DICTIONARY TRIE



We DFS the trie carrying a Levenshtein DP row. When the row's minimum exceeds maxEdits, we prune that whole subtree — pruning we can't do over a flat word list.

SECTION TWO

The Trie.

A tree where edges are *characters* and paths are *prefixes*.

Anatomy of a trie.

DEFINITION

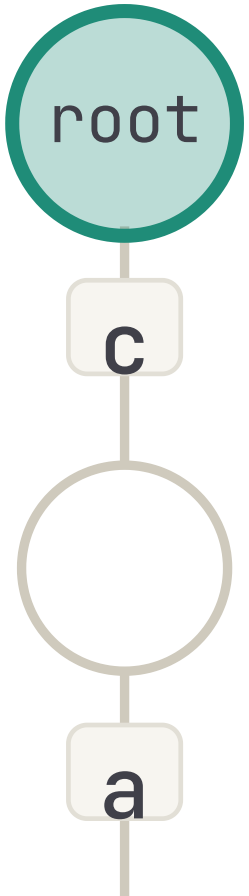
A **trie** (from re*trie*val, pronounced "try") is a rooted tree where:

- Each **edge** is labeled with one character.
- Each **node** represents the prefix formed by the edges from the root to it.
- Some nodes are marked **terminal** — meaning that prefix is a complete inserted word.

WHY A TREE, NOT A HASH MAP?

A hash gives you exact-match in `O(L)`, but loses the *shared prefix structure*. The trie keeps it — that's why prefix queries, wildcards, and edit-distance pruning all become natural.

"CAR", "CARD", "CARE", "CAT"



Two ways to store children.

VERSION A — FIXED ARRAY

Array of size Σ (alphabet)

```
struct Node {
  Node* child[26];   // one slot per 'a'..'z'
  bool  isEnd;
}

// insert
void insert(string s) {
  Node* cur = root;
  for (char c : s) {
    int i = c - 'a';
    if (!cur→child[i])
      cur→child[i] = new Node();
    cur = cur→child[i];
  }
  cur→isEnd = true;
}
```

⊕ O(1) child lookup ⊕ cache-friendly ⊖ Σ pointers per node ⊖ wasteful for unicode

VERSION B — HASH MAP

map<char, Node*> per node

```
struct Node {
  unordered_map<char, Node*> child;
  bool  isEnd;
}

// insert
void insert(string s) {
  Node* cur = root;
  for (char c : s) {
    if (!cur→child.count(c))
      cur→child[c] = new Node();
    cur = cur→child[c];
  }
  cur→isEnd = true;
}
```

⊕ memory ∝ actual children ⊕ any character set ⊖ hash overhead per step ⊖ worse cache behavior

Rule of thumb. Small fixed alphabet (lowercase, DNA, bits) → array. Large or sparse alphabet (Unicode, words as nodes) → map.

Build it yourself.

Type a word and press `insert →` or `search →`, then `next step` to walk one character at a time.

WORD

type a word...

insert → | search → | reset

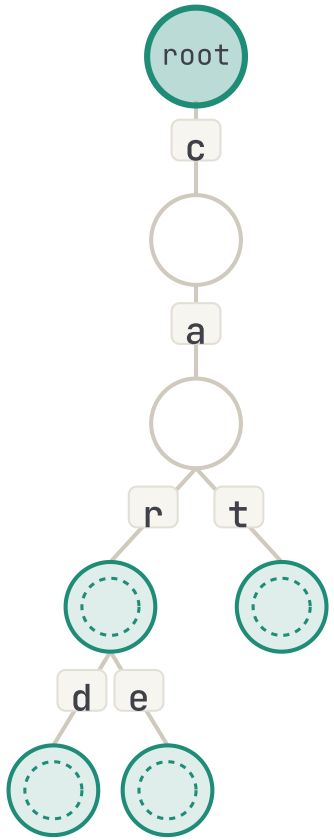
INSERTED (4)

car × | card × | care × | cat ×

QUICK ADD

+ cars | + careful | + cab | + can | + candy

TRIE — TERMINAL NODES ARE DASHED-OUTLINED



Complexity.

OPERATIONS ON A TRIE OF *N* WORDS, MAX LENGTH *L*, ALPHABET Σ

OPERATION	ARRAY VERSION	MAP VERSION	NOTE
insert(w)	O(|w|)	O(|w| · log Σ)	walk down, allocate missing nodes
search(w)	O(|w|)	O(|w| · log Σ)	walk down, check terminal flag
prefixCount(p)	O(|p|)	O(|p| · log Σ)	read passCount at end node
delete(w)	O(|w|)	O(|w| · log Σ)	decrement counts, prune zero-count chains
space	O(N · Σ)	O(N)	N = total nodes ≤ Σ|wᵢ|

BRIEF DERIVATION

For each character of w, we move down by one edge. That's **|w|** steps total. Each step is either:

- **Array:** one indexed access — O(1)
- **Map:** one hash/tree-map lookup — O(1) avg or O(log Σ)

COMPARE TO ALTERNATIVES

DS	SEARCH	PREFIX QUERY
sorted array	O(L log n)	O(L log n + k)
hash set	O(L)	O(n · L)
trie	O(L)	O(L + k)

k = output size

SECTION THREE

Applications.

The same shape, four classic problems.

Prefix occurrences.

Given a set of words, how many start with prefix p? **O(|p|)** if we keep a counter per node.

ADD A WORD

word... add

QUERY — COUNT WORDS WITH PREFIX

car

6

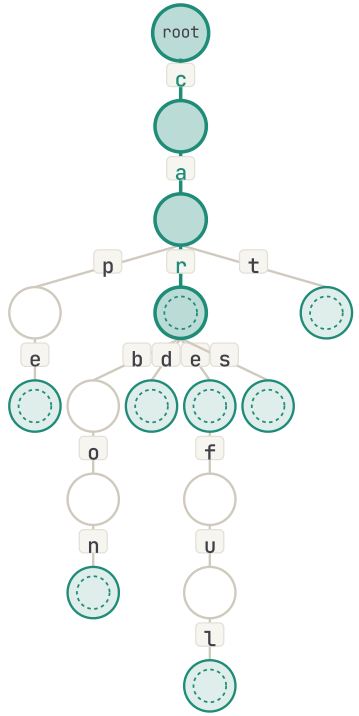
words with prefix "car"

INSERTED (8)

car × card × care × careful × cars × cat × carbon × cape ×

We store `passCount` at each node — incremented on insert. Query is then a single walk: `O(|prefix|)`, no scan of all words.

TRIE — MATCH PATH HIGHLIGHTED, SUBTREE COUNTS VISIBLE



WORDS MATCHED (6)

car carbon card care careful cars

Wildcards, properly.

DFS the trie + the pattern in lockstep. ? branches into all children; * branches at every depth.

PATTERN

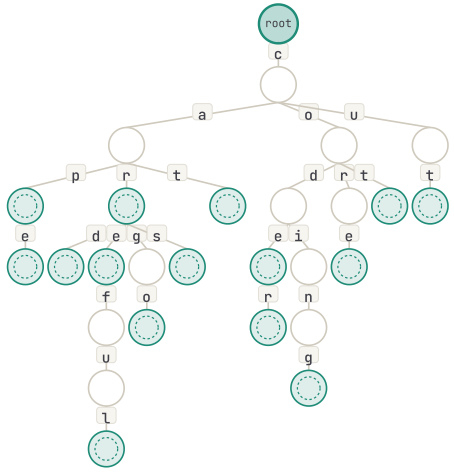
c?r*

? any single char * any sequence

MATCHES (7)

car card care careful cargo cars core

DICTIONARY TRIE



ALGORITHM — PSEUDOCODE

```
match(node, pattern, i):
  if i = |pattern|:
    if node.isEnd: emit(prefix)
    return

  c = pattern[i]

  if c = '?':
    for each child k of node:
      match(node.child[k], pattern, i+1)

  elif c = '*':
    // match zero chars
    match(node, pattern, i+1)
    // match one or more
    for each child k of node:
      match(node.child[k], pattern, i)

  else:
    if node.child[c] exists:
      match(node.child[c], pattern, i+1)
```

On ? we recurse into *every* child; on * we recurse into every subtree at every depth. The trie prunes whole branches that can't match — much faster than scanning all words.

Longest common prefix.

Walk down from root. Stop when a node has more than one child, or is a terminal.

ADD A WORD

word...

add

LONGEST COMMON PREFIX

inter

length: 5

INSERTED (4)

interview × internet × internal × international ×

Walk down from the root. Stop when a node has ≠ 1 child, or it's a terminal. Cost: O(|LCP|).

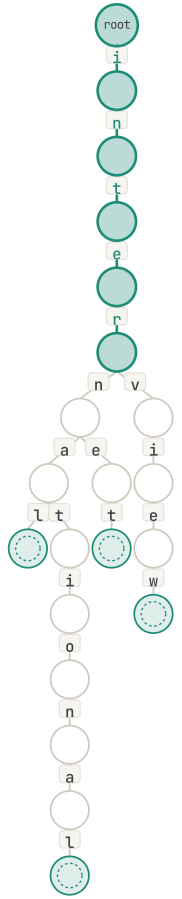
TRY PRESET

flower / flow / flight

no common

nested

TRIE — LCP PATH HIGHLIGHTED



Distinct substrings.

Insert *every suffix* into a trie. The answer is the number of edges = nodes − 1.

INPUT STRING

banana

length: 6

DISTINCT NON-EMPTY SUBSTRINGS

15

ALL SUFFIXES (INSERTED)

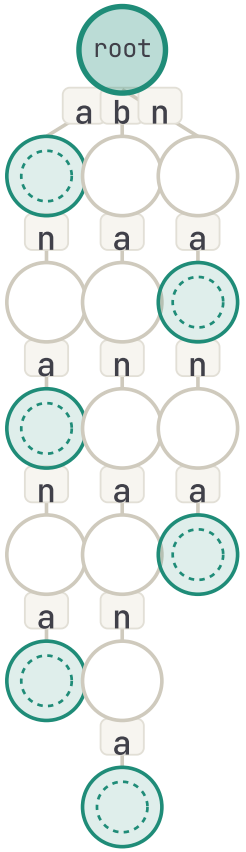
banana | anana | nana | ana | na | a

Insert *every suffix* of the string. The number of distinct non-empty substrings = number of *edges* in the trie = `nodes - 1`. Each path from root corresponds to one distinct substring.

TRY

abab | aaaa | banana | mississip

TRIE OF ALL SUFFIXES



SECTION FOUR

Binary Trie.

Same trie, alphabet = `{0, 1}`. The shape that breaks XOR problems wide open.

Numbers as bit strings.

IDEA

Treat each integer as a fixed-length string of bits (e.g. 5 bits → 0..31, 32 bits → all int). Insert it into a trie with alphabet `{0, 1}`.

Now **every node has at most two children**: one for bit 0 (left) and one for bit 1 (right).

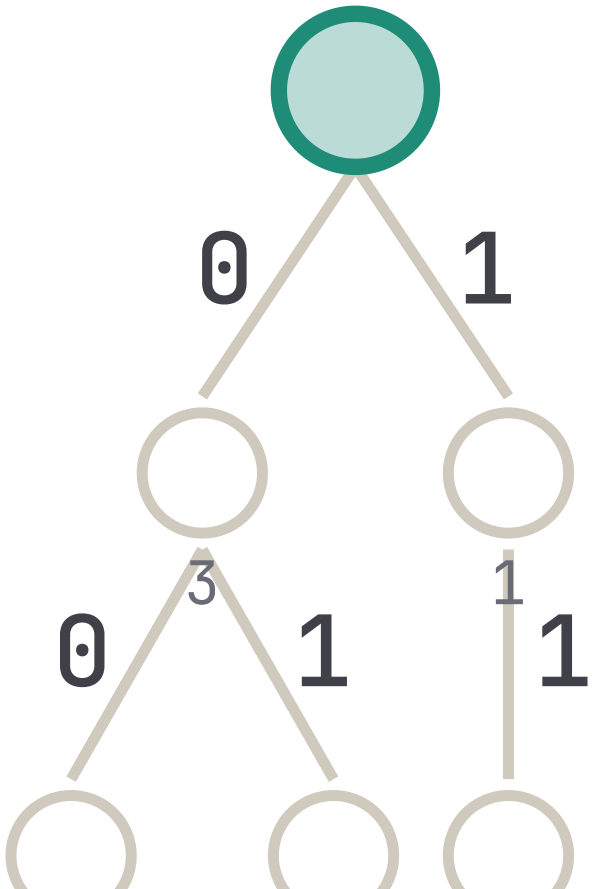
WHY IT'S POWERFUL

XOR is bit-wise. So at each level we can **greedily choose** the branch that flips the current bit (max XOR) or matches it (min XOR). One walk top-to-bottom, `O(B)` per query.

COST

Storing *n* integers of *B* bits each: `O(n · B)` nodes. With `B = 32`, that's tiny.

EXAMPLE — INSERT {3, 6, 12, 25}, B=5



SECTION FIVE

XOR Problems.

Five problems, one data structure.

Max XOR pair.

For each x, walk the trie and prefer the *opposite* bit at every level. O(n · B).

NUMBERS (0..31)

add

3 00011 | 10 01010 | 5 00101 | 25 11001 | 2 00010 | 8 01000

reset | clear

GREEDY WALK FOR QUERY 25

query	1	1	0	0	1	= 25
partner	0	0	1	0	1	= 5
XOR	1	1	1	0	0	= 28

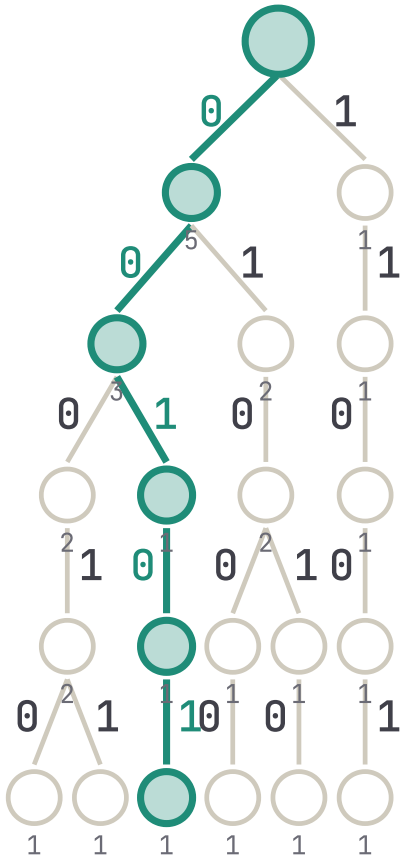
At each level, prefer the opposite bit (greedy) — that maximizes xor.

GLOBAL MAX XOR OVER ALL PAIRS

5 ⊕ 25 = 28

insert each number, then for each x query maxXorWith(x). Total: O(n · B).

BINARY TRIE — GREEN PATH IS THE GREEDY-CHOSEN PARTNER FOR THE QUERY



Min XOR pair.

Mirror trick: prefer the *same* bit at every level — minimize the diff.

NUMBERS

add

3 10 5 25 2 8

SORTED

2 3 5 8 10 25

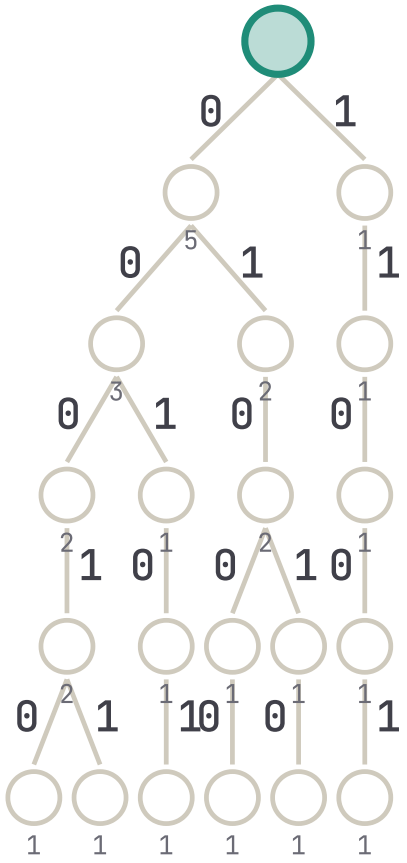
MIN XOR PAIR

2 ⊕ 3 = 1

a	0 0 0 1 0	= 2
b	0 0 0 1 1	= 3
XOR	0 0 0 0 1	= 1

Insight: for any 3 numbers, the minimum-XOR pair is always two of them adjacent in sorted order. So sort + check adjacent pairs in `O(n log n)`. The trie helps when numbers stream in: insert each, then walk down preferring the *same* bit — this lands you at a "neighbor" in trie order.

BINARY TRIE OF ALL NUMBERS



Count XOR pairs.

Use subtree counts. Walk one path, accumulate on branches we won't take.

NUMBERS + K

add num

add

1 4 2 6 5

K = 5

PAIRS WITH XOR < K

6

PAIRS WITH XOR = K

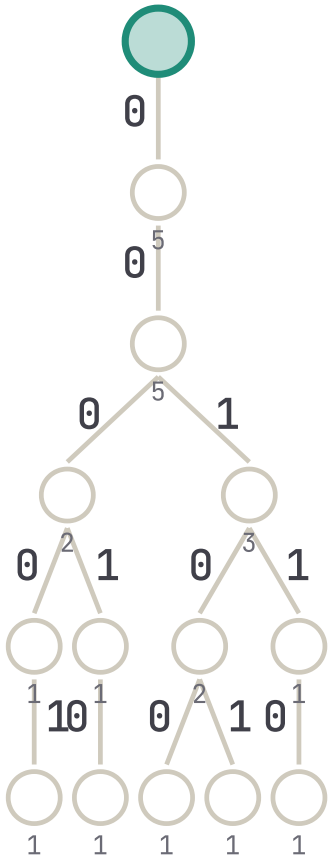
1

ALL PAIRS (BRUTE FORCE, FOR VERIFICATION)

1⊕4=5 1⊕2=3 1⊕6=7 1⊕5=4 4⊕2=6 4⊕6=2 4⊕5=1 2⊕6=4 2⊕5=7 6⊕5=3

Strategy. Process numbers left-to-right. For each new x, count valid partners already in the trie by walking bit-by-bit and using subtree counts — never enumerate pairs. Total: `O(n · B)`.

BINARY TRIE — NODE LABELS SHOW SUBTREE COUNTS



Max XOR subarray.

Reduce to *max XOR pair of prefix XORs*. Insert `P[0]`, then sweep.

ARRAY

add

8 1 2 12 7 6

PREFIX XOR: P[I] = A[0] ⊕ … ⊕ A[I-1]

P0=0 P1=8 P2=9 P3=11 P4=7 P5=0 P6=6

BEST SUBARRAY

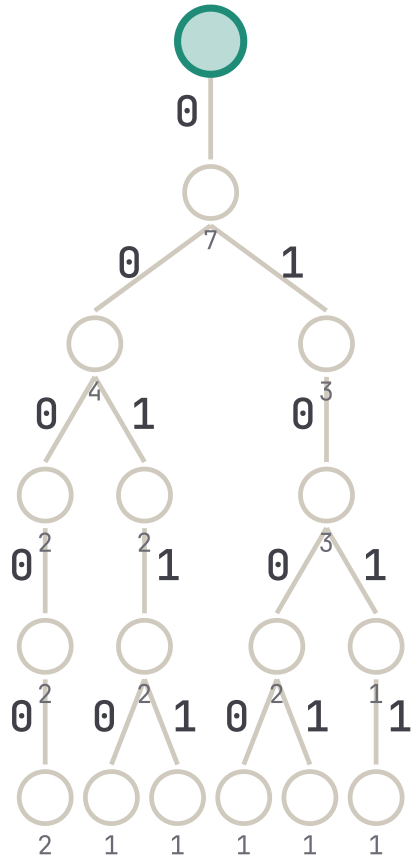
XOR = 15

a[1…3] = [1, 2, 12]

because subarray XOR = P[4] ⊕ P[1] = 7 ⊕ 8 = 15

Trick. XOR of subarray a[j..i-1] = P[i] ⊕ P[j]. So *max subarray XOR* reduces to *max XOR pair among prefix XORs* — exactly the previous problem. `O(n · B)`.

TRIE OF ALL PREFIX XORS



What we learned.

1

Tries store strings as paths

— so any prefix question becomes a single root-to-node walk.

2

Two implementations

— `child[Σ]` for fixed alphabets, `map<c,Node>` for sparse ones.

3

Insert & search are O(L)

— independent of how many words are stored. Pure win over hash for prefix work.

4

Annotate nodes

— `passCount` for prefix counting, `isEnd` for word match, subtree counts for ranking.

5

Wildcards & spell-check

— DFS the trie with the pattern / DP row; prune whole subtrees that can't match.

6

Binary trie ⇒ XOR

— greedy bit-by-bit walk solves max/min XOR, count pairs < K / = K, max XOR subarray.

Whenever you see a problem about **prefixes, character-by-character matching, or bit-by-bit comparisons** — reach for a trie first.

END · LECTURE 07

Questions?

Try every demo at your own pace — every slide is live.

— Ahmed Tawfik

ADSA · Tries · 2026